

Eco Clean – Scheduling Solution

Progress Report 4 – Upul Atapattu

1. Summary

Member	Upul Atapattu
Student ID	300392188

Project Implementation Progress

Since the last progress report, the following has been done.

1.1. DB Schema Revamp

There was a significant database redesign, merging two separate schema versions into one unified model. [Mine and Vidarshan's]. This was the most intensive effort that we have put to stabilize the db schema and It was challenging throughout the period as it needed resolving migration conflicts and the growth of fields and tables to accommodate user interfaces. The database schema was revamped to accommodate all the business functionalities related to the whole web application. This is expected to be the finalized version (in terms of entities in the database schema), which would accommodate the required functionalities and capture data for future decision making. The main entities for different areas are as follows:

Core Business Operations Entities

- **Clients** - The customers that EcoClean serve
- **Jobs** - Work to be done (one-time or recurring basis)
- **Appointments** – These are the scheduled time slots for specific jobs
- **Addresses** - Track the location where jobs will be performed
- **Job Notes** – Information about access, safety, cleaning, etc.
- **Images** attached to appointments, jobs, or notes
- **Visit Notes** - Post-appointment observations
- **Recurrence** – Whether the job is repetitive (daily, weekly, monthly) or one-off

Staff Management Entities

- **Users** – To accommodate different roles (Admin, Staff)
- **Staff Profiles** - This table stores personal employment details
- **Emergency Contacts** and **Staff Addresses** for each employee (Related to the staff profile)
- **Staff Availability** – This tracks when staff are available to work (by day and shift)
- **Leave** – To manage time off of staff

Scheduling & Assignment

- **Assignments** – This link staff to appointments (who is doing what job). This will be the basis for time tracking. Multiple staff can be assigned to the same appointment

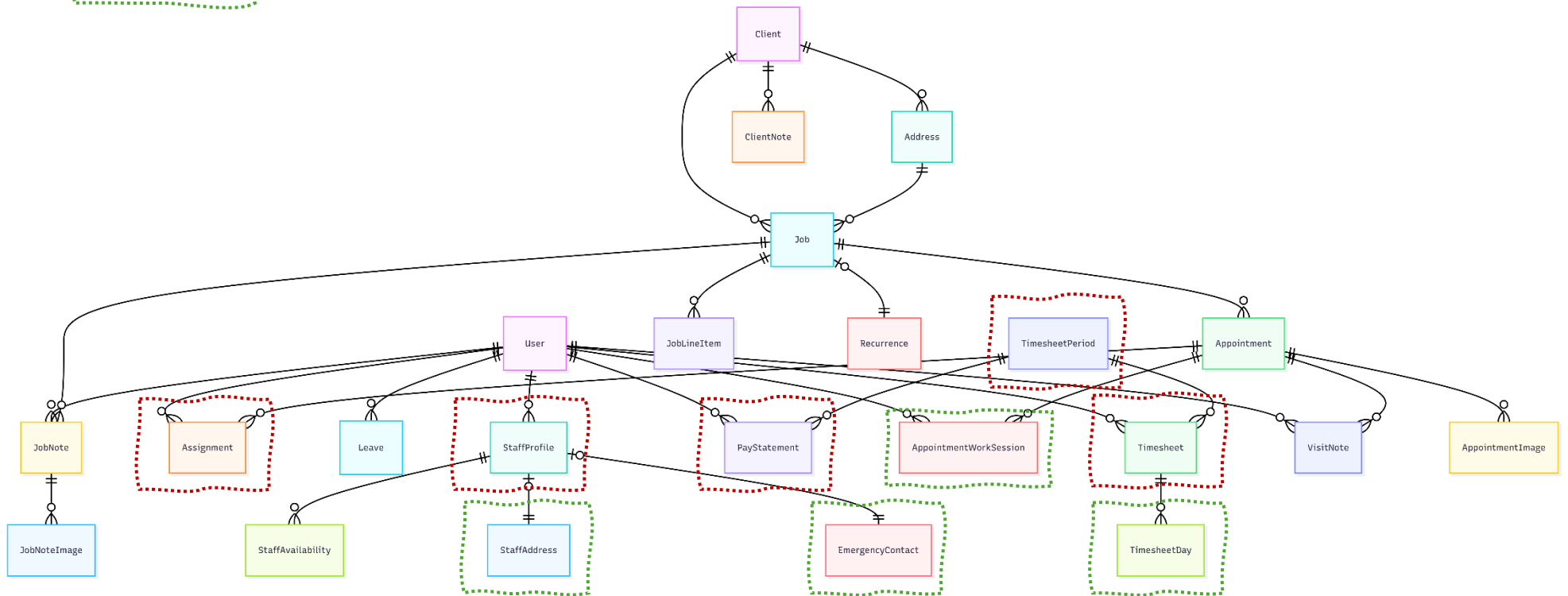
- **Appointment Work Sessions** - Track actual clock-in/clock-out times which is also required for time tracking and via PWA input

Payroll & Time Tracking

- **Timesheet Periods** – This defines pay periods (bi-weekly), which will be providing the basis for timesheets and Pay statements
- **Timesheets** – For each staff member per payment period
- **Timesheet Day** - Tracks the daily hours worked by each staff member
- **Pay Statements** calculate earnings based on $\text{hours} \times \text{rate}$

Revised

Introduced



The previous work done for interfaces and feature implementation used static and sample data to design the user interfaces and functionalities. This period was mainly focused on making it dynamic by integrating the UI and the functions with the database. The staff management and payroll functions were implemented working across the entire stack from database design to frontend implementation. Considering the reliability and functionality that is required, intensive workload has been delivered expecting a cohesive platform which integrate integrates scheduling, time tracking, and payroll handling.

Authentication Issues

In the beginning, authentication was bypassed to reduce the development challenges. However, with the database schema is not stablized, I moved on to fixing persistent authentication issues that had been causing inconsistencies between session data and actual database users. Due to the initial bypass, the development session was showing an admin user while the database had staff records, creating authorization conflicts throughout the system. This required a deep dive into NextAuth configuration, debugging the JWT and session callbacks in both `auth.ts` and `auth-options.ts` by rewriting the `verifyUser` function removing the initial bypass. This work eliminated the development session hacks and established proper role-based access control across the entire application.

User Creation and Staff Profile Backend

The API endpoints for user creation and staff profile management were revamped simultaneously creating entries in User and StaffProfile tables. For the staff profile, a natural key/ID has been introduced STF-ECO-XXXX format to mimic the real practices. This was implemented by nesting Prisma create operations with transaction safety to ensure data consistency. For profile management. Furthermore, both admin-level and staff-level endpoints were added so that any staff member has a staff profile and a dedicated `/api/staff/me` endpoint that lets staff members view and edit their own information.

Job Scheduling and Assignment Logic

The focus was given to the core scheduling logic that connects jobs to staff through appointments and assignments. Working on the jobs API, I preserved the original code structure while enhancing it to handle line items and recurrence patterns. The real complexity came in the appointment and assignment integration, where I built logic to create appointments linked to jobs and then connect staff through assignment records. Each assignment captures planned start and end times at the time of assignment, which is currently linked to timesheet frontend. In the next steps, to ensure historical accuracy for payroll. I also implemented recurring appointment generation, which required careful handling of date calculations and batch creation. Throughout this work, I developed a generic API client with comprehensive error handling that helped debug request-body issues and route-level error propagation between frontend and backend.

Timesheet System and Payroll Infrastructure

The payroll system represented one of the most complex pieces of the project. I built an automated biweekly pay period generation system that can create an entire year's worth of periods with a single server action, following the specific business rule of starting January 1st and ending the first period on the second Saturday. This provides the foundation for all the pay period-related calculations. A dynamic period windows was implemented so that it identify the current date and displays the relevant pay periods (current, previous, and next two periods) while ensuring staff only see their own data through session-based filtering. The most innovative feature here is the integration between scheduled and actual hours, where the system pulls planned shifts directly from the assignment table and presents them alongside input fields for actual hours worked, allowing staff to easily compare what they were scheduled for against what they're logging.

Frontend Development and User Experience

The final piece was bringing everything together in the user interface. Using Mantine UI components consistently across the application, I developed three main pages. The staff profile page features editable fields for contact information and address details alongside read-only fields for name and staff ID, with a complete edit/save/cancel workflow and proper loading states. The pay management page dynamically maps the 14-day periods with intelligent labeling that identifies dates and marks today's date, all while maintaining the Mantine design language. The enter time interface presents scheduled hours from assignments alongside input fields for actual hours, creating a clear visual comparison. Throughout the frontend work, I implemented comprehensive error handling and user feedback mechanisms to ensure a smooth experience even when things go wrong.

1.2. Next Steps

The following are the tasks which are planned by the next progress report deadline (23rd March 2026):

- **Finalize Payroll Management Flow**

Complete the development and testing of payroll related interfaces, corresponding API endpoints, and database CRUD operations, with the objective of stabilizing these features by the next progress report.

- **Support Role Based Access**

Many of the interfaces in my branches are not structured in a way to support role based access. In the next few days this is expected to be sorted out.

- **Merge the current development branch with Vidarshan's branch in GitHub**

Resolve any build or TypeScript-related issues that arise during the integration process to ensure a stable and consistent codebase, ready to Vercel Deployment

- **Real life Test Data Generation**

Populate the data with the support with EcoClean to thoroughly test system functionality, validate workflows, and identify potential edge cases across the application.

- **Settings Page to Customize the solution for different companies**

Though this solution is catered to by EcoClean, we believe this solution can resolve challenges faced by many small service companies. Therefore, some effort will be made towards customizing the solution through settings.

2. Worklogs

Date	Hours	Description
15th January to 9th March	78.25 hours	Previous Work
10th March 11:00 pm – 12:00 pm	1 hour	Worked on debugging on some of the UI components related to the payroll.
12th March 6:00 pm – 7:00 pm 12:30 pm – 2:30 pm 10:30 pm – 11:30 pm	4 hours	Introduced StaffProfile, StaffAddress, and EmergencyContact entities with proper one-to-one relationships. Built frontend Staff Profile page using Mantine UI with editable form fields for contact information, address, and emergency contact. Implemented Edit/Save/Cancel workflow with loading states. Debugged authentication inconsistencies between session user and database user. Investigated NextAuth JWT and session callbacks, resolved Git merge conflicts in lib/auth.ts, and rewrote verifyUser function to properly integrate with Prisma and bcrypt. Implemented proper session-to-user mapping using real User.id and role, removing development session hacks. Tested authentication flow across different user roles to ensure proper access control.
13th March 9:30 am – 10:30 am 6:00 pm – 7:00 pm	2 hours	Built user creation API with conditional StaffProfile generation for STAFF role users. Implemented automatic generation of temporary passwords and natural staff IDs (STF-ECO-XXXX format). Used Prisma nested create operations with transaction safety to ensure data consistency. Refactored logic to use session-based user identification instead of hardcoded IDs. Tested profile updates with transaction-based Prisma operations.
14th March 8:30 pm – 10:00 pm	1.5 hours	Developed comprehensive staff profile API endpoints including GET /api/staff/[id], GET /api/staff/me, and

		PATCH functionality. Implemented role-based access control with TypeScript fixes for nested relation includes.
15th March 9:30 am – 10:30 am 6:00 pm – 7:00 pm 11:00 pm – 1:00 am	4 hours	Continued job scheduling work, implementing recurring appointment generation logic. Handled complex date calculations and batch creation for recurring assignments. Created automated biweekly pay period generation system with "run-once" server action to generate full year of periods. Implemented Prisma transactions for atomic batch creation following business rules of starting Jan 1st and ending first period on second Saturday. Built dynamic period windows that identify current date and display relevant pay periods with session-based security filtering. Developed Pay Management UI with dynamic date mapping for 14-day periods and intelligent labeling.
16th March 8:30 am – 10:00 pm 3:00 pm – 4:30 pm 6:00 pm – 7:00 pm 8:00 pm-9:30 pm	5.5 hours	Created Enter Time interface that pulls scheduled hours from assignments alongside input fields for actual hours. Implemented visual comparison between scheduled and logged time with session-based data filtering. Undertook a major database schema revamp to merge two separate schema versions into one unified model. The goal was to integrate the appointment work sessions and authored job notes from the original schema with the assignment-based staffing and comprehensive payroll system from the newer version. This involved careful analysis of both schemas to identify overlapping entities and establish proper relationships without data loss. Worked on creating the progress report and updating the AI section and worklogs.
Total	96.25 hours	

3. AI Usage (From 10th March onwards)

Please note that only the principal prompt is included in the table. In certain cases, I had put the entire error message or code, in case I was not able to pinpoint the error/issue.

AI	Prompt	Value Addition
Gemini, Pro	I'm going to put payPeriodStart DateTime payPeriodEnd DateTime in my timesheetEntry. So I need to get it from TimesheetEntry to my paystatemetn. How to write the relationship., payPeriod supposed to be 5+date from payPeriodEnd	To Modify the PayStatement Table
Gemini, Pro	Employ name to be picked up based on the UserID, which is to be taken from the auth session. How do add this to this code?	To Understand how to get the UserID from a auth session

Gemini, Pro	I'm getting the following error. Lets fix that <pre> <Box> 124 <Text fw={600} mb={10}>Weekly Shift Availability *</Text> > 125 <Card withBorder radius="md" p={0}> ^ 126 <Table verticalSpacing="sm" horizontalSpacing="md"> 127 <Table.Thead> </pre>	Debugging UI Interfaces
Gemini, Pro	I want to create page where we have a form to update payperiod details as per the image. How should I approach this?	Designing the UserInterface for PayStubs
Gemini, Pro	In my paystubs page, I want to display So, when I'm trying the route.ts, how should I design. What should be the path?	To retrieve the data to generate the charts and pay summary

Gemini, Pro	In my staff profile section, I need to edit my staff profile details. How can I change my staff profile Read only fields to Read/Input fields? Example, <pre><TextInput value={value} readOnly styles={{ input: { height: 44 } }} w="100%" /></pre>	To Make the Staff Profile more editable
ChatGPT, Free	Type '{ staffProfile ? : { create: { staffId: string; name: string; hourlyRate: number; }; } undefined; name: string; email: string; role: Role; password: string; }' is not assignable to type '(Without<UserCreateInput, UserUncheckedCreateInput> & UserUncheckedCreateInput) (Without <...> & UserCreateInput)'. Type '{ staffProfile ? : { create: { staffId: string; name: string; hourlyRate: number; }; } undefined; name: string; email: string; role: Role; password: string; }' is not assignable to type 'Without<UserUncheckedCreateInput, UserCreateInput> & UserCreateInput'. Types of property 'staffProfile' are incompatible. Type '{ create: { staffId: string; name: string; hourlyRate: number; }; } undefined' is not assignable to type 'StaffProfileCreateNestedOneWithoutUserInput undefined'. Type '{ create: { staffId: string; name: string; hourlyRate: number; }; }' is not assignable to type 'StaffProfileCreateNestedOneWithoutUserInput'. create' are incompatible. property 'cr' Types of pr Why am I getting this error?	Debugging

ChatGPT, Free	I have updated my prisma scheme as follows: <pre> model StaffProfile { id String @id @default(uuid()) staffId String @unique userId String @unique user User @relation(fields: [userId], references: [id]) postalCode String? hourlyRate Float @default(0) availabilities StaffAvailability[] } </pre> Then I got this error when trying to create a user <pre> Invalid `{{imported module ./lib/prisma.ts}}["prisma"].user.create()` invocation in C:\Douglas\AppliedResearch\W26_4495_S2_UpulA\Implementation\eco- clean-web\.next\dev\server\chunks\[root-of-the- server]__823bdd2d_.js:375:151 372 const randomSuffix = Math.random().toString(36).substring(2, 373 const staffId = `STF-ECO-\${randomSuffix}`; 374 const minWage = 18.5; → 375 const user = await {{imported module ./lib/prisma.ts}}["prisma"].user.create({ data: { name: "Ayesha Kumarasena", email: "ayeshakumarasena@gmail.com", role: "STAFF", password: "\$2b\$10\$vt.GwCnJH6OyRMYqKtNmudJFV28mrPA7Mo.nW0wZ1KYI7t GsCMHG", staffProfile: { create: { staffId: "STF-ECO-TILI", ~~~~~ hourlyRate: 18.5, ? id?: String, ? postalCode?: String Null, ? availabilities?: StaffAvailabilityCreateNestedManyWithoutStaffProfileInput } } } }) } </pre>	
--------------------------	--	--

	<pre> } }, select: { id: true, name: true, email: true, role: true, createdAt: true } }) </pre> Unknown argument `staffId`. Available options are marked with ?. <pre> at <unknown> (app\api\users\route.ts:128:34) at async POST (app\api\users\route.ts:128:16) 126 const minWage = 18.5; 127 > 128 const user = await prisma.user.create({ ^ 129 data: { name, email, role, password: hashedPassword, 130 131 ...(role === "STAFF" && { { clientVersion: '7.4.1' } } POST /api/users 500 in 295ms </pre>	
ChatGPT, Free	When I create a job, in prisma studio it goes as _AppointmentStaff for What does it mean?	To understand the initial db behaviour
ChatGPT, Free	Keepin the _AppointmentStaff as it is, how can I add the entry to my assigment table?	To to POST rows in the Assignment table in db

ChatGPT, Free	I'm confused about changing it in the recurring loop if (jobType === 'RECURRING') { const { frequency, interval, endType, endsAfter, endsOn } = recurrence!; // Store recurrence row await tx.recurrence.create({ data: { jobId: job.id, frequency, interval, endType, endsAfter: endType === 'after' ? (endsAfter ?? null) : null, endsOn: endType === 'on' && endsOn ? new Date(endsOn) : null, }, });	To debug Assignment CRUD route.ts functions
ChatGPT, Free	Cannot find name 'apptStart'.ts(2304), What is this error?	To debug Assignment CRUD route.ts functions
ChatGPT, Free	I want to remove the id from here and get the id from the session. How can I adjust this in the following code? try { // In a real app, you'd get the ID from the session. // For testing, use the ID you used in Postman. const response = await fetch('/api/staff/3b32d468-9f20-4808-9f25-bffabed6a9cb',); const result = await response.json();	To remove the fixed id I have used when establishing the UI, but, now I want to do the session based data fetching
ChatGPT, Free	How can I change this kind a '/api/staff/3b32d468-9f20-4808-9f25-bffabed6a9cb' url adjustment to some other way which doesn't need the user id on the url?	To remove this url based api routing
ChatGPT, Free	GET / 307 in 2.3s (compile: 1678ms, proxy.ts: 263ms, render: 392ms) ✕ ./lib/auth.ts:4:1 Parsing ecmascript source code failed 2 import bcrypt from "bcrypt"; 3 > 4 <<<<<<< Updated upstream ^^^^^^^ 5 export async function verifyUser(email: string, password: string) { 6 const normalizedEmail = email.toLowerCase().trim(); 7 Merge conflict marker encountered. What is this error?	For Debugging after Git Merging. Successfully remove the conflict marker after learning this
ChatGPT, Free	What is the error? lib/api/client.ts (111:11) @ apiClient 109 if (!res.ok) { 110 const err = (data as ApiErrorShape null)?.error; > 111 throw new Error(err "Request failed"); ^ 112 } 113 114 return data as TResponse;	For Debugging frontend UI

ChatGPT, Free	How can I adjust this code, keep the original code as much as possible and adjust the Assignment insert ? <pre> export const runtime = 'nodejs'; import { prisma } from '@lib/prisma'; import { NextRequest, NextResponse } from 'next/server'; import { AppointmentStatus, JobType } from '@prisma/client'; // Infer tx type from prisma.\$transaction callback signature type Tx = Parameters<Parameters<typeof prisma.\$transaction>[0]>[0]; type IncomingBody = string { title?: string; clientId?: string; addressId?: string; jobType?: JobType; // "ONE_OFF" "RECURRING" isAnytime?: boolean; visitInstructions?: string null; lineItems?: Array<{ id?: string; // ignored name?: string; quantity?: number; unitCost?: number; unitPrice?: number; description?: string null; }>; appointments?: Array<{ id?: string; // ignored // you send ISO datetime like "2026-02-25T08:00:00.000Z" startDate?: string; startTime?: string null; // "07:00" endTime?: string null; // "09:30" staffId?: string[]; // ["userId1", ...] notes?: string null; images?: Array<{ url: string; fileKey?: string }>; }>; recurrence?: { frequency: 'weekly' 'monthly'; interval: number; endType: 'after' 'on'; endsAfter?: number null; endsOn?: string null; // "YYYY-MM-DD" } null; }; type Body = { title?: string; clientId?: string; staffId?: string null; addressId?: string null; jobType?: 'ONE_OFF' 'RECURRING'; startDate?: string; // e.g. "2026-02-25" startTime?: string null; // e.g. "13:30" endTime?: string null; // e.g. "15:00" isAnytime?: boolean; recurrence?: { frequency: 'weekly' 'monthly'; interval: number; endType: 'after' 'on'; endsAfter?: number null; endsOn?: string null; // "YYYY-MM-DD" } null; visitInstructions?: string null; }; </pre>	To insert an row simultaneously together with _AssignmentStuff
ChatGPT, Free	My table is cut-off. I want this to be expanded for the entire width, how can I do that? <pre> if (loading) return (<Container py="xl" ta="center"> <Loader size="xl" /> </Container>); return (<Container size="lg" py="xl"> <Title order={2} mb="xl"> Payment History </Title> <Card withBorder radius="md" p="md"> /Card> </Container>); } </pre>	I adjusted the pay-period-management UI
ChatGPT, Free	How does a real-life timesheet submission work? Let's say, a worker submit the timesheet, then there is verification by the supervisor and then it will go to payroll, right? What is the common practice. To keep most detailed job data(hours per day) with the worker? then reducing it to days which can be approved by the supervisor and payroll having the least visibility?	To understand the flow of how I should design the time sheet approval and adjust the db design

ChatGPT, Free	If I do my Timesheet as model TimesheetEntry { id String @id @default(uuid()) periodId String staffId String W1Sun W1Mon W1Tue ... W2Sat} What if I modify the db schema like this. When I submit a biweekly timesheet, all the daily data will be saved. Then the supervisor who gets this can verify this against daily planned activity, right?	To design the Timesheet related entities in the database
ChatGPT, Free	How can I do Timesheet Entries in a more detailed manner keeping the daily level information	To design the Timesheet related entities in the database

ChatGPT, Free	<pre>Schema1 (Give Priority) model User { id String @id @default(uuid()) name String email String @unique role Role createdAt DateTime @default(now()) password String assignedAppointments Appointment[] @relation("AppointmentStaff") workSessions AppointmentWorkSession[] jobNotesAuthored JobNote[] @relation("JobNoteAuthor") visitNotesAuthored VisitNote[] @relation("VisitNoteAuthor") } enum Role { ADMIN STAFF CLIENT } enum JobType { ONE_OFF RECURRING } enum AppointmentStatus { SCHEDULED COMPLETED CANCELLED LATE } Schema2 generator client { provider = "prisma-client-js"</pre>	Merging the finalized DB Design and Debugging
------------------	---	---

```

}

datasource db {
  provider = "postgresql"
}

model User {
  id          String      @id @default(uuid())
  name        String
  email       String      @unique
  role        Role
  createdAt   DateTime    @default(now())
  password    String
  assignedAppointments Appointment[] @relation("AppointmentStaff")
  staffProfile StaffProfile?

  .....
model EmergencyContact {
  id          String      @id @default(uuid())
  staffProfileId String    @unique
  name        String
  phoneNumber  String
  relationship String

  staffProfile StaffProfile @relation(fields: [staffProfileId], references: [id],
onDelete: Cascade)
}

model Assignment {
  id          String      @id @default(uuid())
  appointmentId String
  staffId     String
  status      AssignmentStatus @default(PENDING)

  // Timing & Travel
  plannedStart DateTime? // Includes estimated travel
  plannedEnd   DateTime?
  actualStart  DateTime? // Captured via staff clock-in
  actualEnd    DateTime? // Captured via staff clock-out

```

```

breakMinutes Int    @default(0)

// Payroll Snapshots
hourlyRateAtTime Float? // Lock the rate in case staff gets a raise later

notes String?
createdAt DateTime @default(now())
updatedAt DateTime @updatedAt

// Relations
appointment Appointment @relation(fields: [appointmentId],
references: [id], onDelete: Cascade)
staff User @relation(fields: [staffId], references: [id])
}

model StaffAvailability {
  id String @id @default(uuid())
  staffProfileId String
  effectiveFrom DateTime
  createdAt DateTime @default(now())

  // Monday
  monActive Boolean @default(false)
  monS1 Boolean @default(false) // Morning Shift
  monS2 Boolean @default(false) // Afternoon Shift

  // Tuesday
  tueActive Boolean @default(false)
  tueS1 Boolean @default(false)
  tueS2 Boolean @default(false)

  // Wednesday
  wedActive Boolean @default(false)
  wedS1 Boolean @default(false)
  wedS2 Boolean @default(false)

  // Thursday
  thuActive Boolean @default(false)
  thuS1 Boolean @default(false)

```

```

thuS2      Boolean  @default(false)

// Friday
friActive   Boolean  @default(false)
friS1       Boolean  @default(false)
friS2       Boolean  @default(false)

// Saturday
satActive   Boolean  @default(false)
satS1       Boolean  @default(false)
satS2       Boolean  @default(false)

// Sunday
sunActive   Boolean  @default(false)
sunS1       Boolean  @default(false)
sunS2       Boolean  @default(false)

staffProfile StaffProfile @relation(fields: [staffProfileId], references: [id],
onDelete: Cascade)
}

model Leave {
  id      String  @id @default(uuid())
  staffId String
  type    LeaveType
  startAt DateTime
  endAt   DateTime
  reason  String?
  createdAt DateTime @default(now())

  staff User @relation(fields: [staffId], references: [id])
}

model TimesheetPeriod {
  id      String  @id @default(uuid())
  startDate DateTime
  endDate  DateTime
  status   TimesheetPeriodStatus @default(OPEN)
  createdAt DateTime @default(now())

```

```

lockedAt DateTime?
timesheets Timesheet[]

payStatements PayStatement[]
}

model Timesheet {
  id      String      @id @default(uuid())
  periodId String
  staffId  String
  status   TimesheetStatus @default(OPEN)
  submittedAt DateTime?
  approvedAt DateTime?
  approvedBy String?
  notes    String?
  createdAt DateTime    @default(now())
  updatedAt DateTime    @updatedAt

  period TimesheetPeriod @relation(fields: [periodId], references: [id],
onDelete: Cascade)
  staff   User           @relation(fields: [staffId], references: [id], onDelete:
Cascade)
  days    TimesheetDay[]

  @@unique([periodId, staffId])
  @@index([staffId])
  @@index([periodId])
}

model TimesheetDay {
  id      String @id @default(uuid())
  timesheetId String
  workDate DateTime
  minutesWorked Int
  hourlyRate Float?
  notes    String?
  createdAt DateTime @default(now())

  timesheet Timesheet @relation(fields: [timesheetId], references: [id],

```

```

onDelete: Cascade)

@@unique([timesheetId, workDate])
@@index([workDate])
}

model PayStatement {
  id          String  @id @default(uuid())
  userId      String
  timesheetPeriodId String

  // Core Dates
  payPeriodStart DateTime
  payPeriodEnd   DateTime
  payDate        DateTime

  // Core Financials
  grossEarnings  Float
  totalDeductions Float
  netEarnings    Float

  // Flat Line Item Data (Optional: store breakdown as a JSON string)
  // This replaces the separate "PayStatementLine" model entirely
  breakdown      Json?

  // Relations
  user           User      @relation(fields: [userId], references: [id],
onDelete: Cascade)
  timesheetPeriod TimesheetPeriod @relation(fields: [timesheetPeriodId],
references: [id], onDelete: Cascade)

  @@index([userId])
  @@index([timesheetPeriodId])
}

enum PayLineCategory {
  EARNING
  DEDUCTION
}

```

```
enum Role {  
  ADMIN  
  STAFF  
  CLIENT  
}
```

```
enum JobType {  
  ONE_OFF  
  RECURRING  
}
```

```
enum AppointmentStatus {  
  SCHEDULED  
  COMPLETED  
  CANCELLED  
  LATE  
}
```

```
enum AssignmentStatus {  
  PENDING  
  EN_ROUTE  
  ON_SITE  
  COMPLETED  
  SICK_LEAVE  
}
```

```
enum LeaveType {  
  PAID_SICK  
  VACATION  
  PERSONAL  
  UNPAID_SICK  
}
```

```
enum TimesheetStatus {  
  OPEN  
  SUBMITTED  
  APPROVED  
  LOCKED
```


}

```
enum TimesheetPeriodStatus {  
  OPEN  
  LOCKED  
}
```

Can you merge schema 1 and schema 2, preserving the schema 1 and its relations

ChatGPT, Free	Type '{ staffProfile?: { create: { staffId: string; name: string; hourlyRate: number; }; } undefined; name: string; email: string; role: Role; password: string; }' is not assignable to type '(Without<UserCreateInput, UserUncheckedCreateInput> & UserUncheckedCreateInput) (Without <... > & UserCreateInput)'. Type '{ staffProfile?: { create: { staffId: string; name: string; hourlyRate: number; }; } undefined; name: string; email: string; role: Role; password: string; }' is not assignable to type 'Without<UserUncheckedCreateInput, UserCreateInput> & UserCreateInput'. Types of property 'staffProfile' are incompatible. Type '{ create: { staffId: string; name: string; hourlyRate: number; }; } undefined' is not assignable to type 'StaffProfileCreateNestedOneWithoutUserInput undefined'. Type '{ create: { staffId: string; name: string; hourlyRate: number; }; }' is not assignable to type 'StaffProfileCreateNestedOneWithoutUserInput'. create' are incompatible. property 'cr Types of pr Why am I getting this error? <pre> export async function POST(req: Request) { const body = (await req.json()) as { name?: string; email?: string; role?: Role; }; const { name, email, role } = body; if (!name !email !role) { return NextResponse.json({ error: "Missing required fields" }, </pre>	To debug inserting User rows simultaneously with Staff
--------------------------	--	---

```
    { status: 400 },
  );
}

const tempPassword = generatePassword(14);
const hashedPassword = await bcrypt.hash(tempPassword, 10);

// 1. Generate the natural key suffix (4 alphanumeric characters)
const randomSuffix = Math.random().toString(36).substring(2,
6).toUpperCase();
const staffId = `STF-ECO-${randomSuffix}`;

const user = await prisma.user.create({
  data: {
    name,
    email,
    role,
    password: hashedPassword,
    // 2. Conditionally create StaffProfile ONLY if the role is STAFF
    ...(role === "STAFF" && {
      staffProfile: {
        create: {
          staffId: staffId, // The natural key
          name: name,      // Inheriting the name from User
          hourlyRate: 0,   // Default value
        }
      }
    })
  },
  select: {
    id: true,
    name: true,
    email: true,
    role: true,
    createdAt: true,
    staffProfile: true, // Include the profile in the response to verify
  },
});
```

```
return NextResponse.json({ user, tempPassword }, { status: 201 });  
}
```

Why am I getting this error?

ChatGPT, Free	If the npx prisma migrate dev doesn't work because the current data overlap, should I drop the database/reset the database? How to do that?	To reset the db after significant changes in the DB design
ChatGPT, Free	What is the error? Unknown argument staffId. Available options are marked with ?. ✖ Error [PrismaClientValidationError]: Invalid {imported module ./lib/prisma.ts}["prisma"].user.create() invocation in C:\Douglas\AppliedResearch\W26_4495_S2_UpulA\Implementation\eco-clean-web\.next\dev\server\chunks\[root-of-the-server]__823bdd2d._.js:375:151 372 const randomSuffix = Math.random().toString(36).substring(2, 6).toUpperCase(); 373 const staffId = STF-ECO-\${randomSuffix}; 374 const minWage = 18.5; → 375 const user = await {imported module ./lib/prisma.ts}["prisma"].user.create({ data: { name: "Ayesha Kumarasena", email: "ayeshakumarasena@gmail.com", role: "STAFF", password: "\$2b\$10\$QQlsHZOUfdl3mAJEZeFoEeKsNVHI0RxLYMq6/Nzx5vF23w3OgeBOG",	To debug when updating my post function to match my current db schema.
ChatGPT, Free	how to fix Type '{ id: string; staffId: string; hourlyRate: number; userId: string; position: string null; }' is not assignable to type '{ staffAddress: { id: string; createdAt: Date; staffProfileId: string; street1: string; street2: string null; city: string; province: string; postalCode: string null; country: string; } null; emergencyContact: { ...; } null; } & { ...; }'. Type '{ id: string; staffId: string; hourlyRate: number; userId: string; position: string null; }' is missing the following properties from type '{ staffAddress: { id: string; createdAt: Date; staffProfileId: string; street1: string; street2: string null; city: string; province: string; postalCode: string null; country: string; } null; emergencyContact: { ...; } null; }': staffAddress, emergencyContactts(2322)	Debugging StaffProfile post function

ChatGPT, Free	Unknown argument `staffId`. Available options are marked with ?. ✖ Error [PrismaClientValidationError]: Invalid `{imported module ./lib/prisma.ts}["prisma"].user.create()` invocation in C:\Douglas\AppliedResearch\W26_4495_S2_UpulA\Implementation\eco-clean-web\.next\dev\server\chunks\[root-of-the-server]__823bdd2d.__.js:369:151 <pre>366 const randomSuffix = Math.random().toString(36).substring(2, 6).toUpperCase(); 367 const staffId = `STF-ECO-\${randomSuffix}`; 368 const minWage = 18.5; → 369 const user = await {imported module ./lib/prisma.ts}["prisma"].user.create({ data: { name: "Ayesha Kumarasena", email: "ayeshakumarasena@gmail.com", role: "STAFF", password: "\$2b\$10\$68Ffhbg6SyV7.rVJpMa6Zu9pG.4B4BrZ.wGEiR5ZxytTgFnHbDSl K", staffProfile: { create: { staffId: "STF-ECO-CE72", ~~~~~ hourlyRate: 18.5, position: null, ? id?: String, ? postalCode?: String Null, ? availabilities?: StaffAvailabilityCreateNestedManyWithoutStaffProfileInput } }, }, select: { id: true, name: true, email: true, role: true,</pre>	Debugging StaffProfile post function
------------------	--	--------------------------------------

```
    createdAt: true,  
    staffProfile: {  
      select: {  
        id: true,  
        staffId: true,  
        position: true,  
        hourlyRate: true  
      }  
    }  
  }  
})
```

Unknown argument `staffId`. Available options are marked with ?.

at <unknown> (app\api\users\route.ts:120:34)

at async POST (app\api\users\route.ts:120:16)

```
118 |   const minWage = 18.5;
```

```
119 |
```

```
> 120 |   const user = await prisma.user.create({  
      |               ^
```

```
121 |     data: {
```

```
122 |       name,
```

```
123 |       email, {
```

```
    clientVersion: '7.4.1'
```

```
  }
```

What is the error???

ChatGPT, Free	How to fix app/(dashboard)/staff-profile/page.tsx (65:67) @ fetchProfile <pre> 63 id: result.id, 64 name: result.name, > 65 staffIdDisplay: result.staffProfile?.staffId result.id.slice(0, ^ 66 phone: result.phone "", 67 address: result.staffProfile?.staffAddress?.street1 "", 68 postalCode: result.staffProfile?.staffAddress?.postalCode "", </pre>	Debugging StaffProfile post function
ChatGPT, Free	What is this error? page.tsx:59 GET http://localhost:3000/api/staff/me 404 (Not Found) fetchProfile @ page.tsx:59 StaffProfilePage.useEffect @ page.tsx:54 "use client" Promise.all @ VM842 <anonymous>:1 Promise.all @ VM842 <anonymous>:1 Promise.all @ VM842 <anonymous>:1	fetchProfile function did not work with the userID, as the dev session had a authentication bypass. To figure this out
ChatGPT, Free	GET /api/auth/session 200 in 279ms (compile: 51ms, proxy.ts: 73ms, render: 155ms) SESSION: { user: { name: 'Developer', email: 'ayeshakumarasena@gmail.com', image: undefined, id: 'dev-user', role: 'ADMIN' } } SESSION USER ID: dev-user SESSION USER ROLE: ADMIN GET /api/users?paginate=true&page=1&limit=20 200 in 418ms (compile: 22ms, proxy.ts: 73ms, render: 323ms) GET /api/clients?q= 200 in 430ms (compile: 14ms, proxy.ts: 76ms, render: 340ms) DB USER: null GET /api/staff/me 404 in 407ms (compile: 162ms, proxy.ts: 21ms, render: 225ms) how to fix this?	fetchProfile function did not work with the userID, as the dev session had a authentication bypass. To figure this out

4. References

- BullMQ - Background Jobs processing and message queue for NodeJS. (2018). Bullmq.io. <https://bullmq.io/>
- Expo. (n.d.). Expo. Expo. <https://expo.dev/>
- Home - Nettoyage Eco Vert. (2026, January 3). Nettoyage Eco Vert. <https://nettoyageecovert.ca/>
- Mantine. (2025, May 5). *Version 8.0.0* (Changelog). Mantine.dev. <https://mantine.dev/changelog/8-0-0/>
- Prisma. (n.d.). *Introduction to Prisma*. Prisma Documentation. <https://www.prisma.io/docs>
- Redis. (n.d.). Redis. Redis.io. <https://redis.io/>
- Vercel. (2026, February 20). *Getting started (App Router)*. Next.js Documentation. <https://nextjs.org/docs/app/getting-started>
- Supabase. (n.d.). *Supabase – The open-source Firebase alternative*. <https://supabase.com/>